

Муромский институт (филиал) ВлГУ

Факультет информационных технологий и радиоэлектроники

Кафедра программной инженерии

КУРСОВАЯ РАБОТА

по дисциплине «Разработка приложений для мобильных операционных систем»

Мобильное приложение «Гостиница»

Пояснительная записка · версия 0.9.0

Выполнил: Арзимуротов А.М., группа ПИН-123

Муром, 2026

Содержание

Аннотация	4
Abstract	5
Введение	6
1. Анализ технического задания	7
1.1. Предметная область и границы системы	7
1.2. Роли и пользовательские сценарии	8
1.3. Обзор аналогов	9
1.4. Требования и критерии корректности	10
2. Разработка алгоритмов	12
2.1. Архитектура итоговой системы	12
2.2. Структуры данных и SQL-схема	14
2.3. Алгоритм поиска и доступности	16
2.4. Атомарное создание заказа	17
2.5. Состояния заказа и демооплаты	20
2.6. Аккаунты и восстановление доступа	20
2.7. Администрирование, фотографии и карты	21
2.8. HTML-документ и передача почтовому клиенту	22
2.9. Подход к тестированию	22
2.10. Выводы по главе	23
3. Руководство программиста	24
3.1. Среда разработки и структура проекта	24
3.2. Сборка и запуск на Mac и Asus	24
3.3. Изменение кода и SQL-схемы	25
3.4. Документация и контроль версий	26
4. Руководство пользователя	27

4.1. Установка и первый запуск	27
4.2. Вход, регистрация и пароль	27
4.3. Поиск и оформление проживания	28
4.4. Кабинет, подтверждение и почта	28
4.5. Места, карта и администрирование	29
Заключение	30
Список используемой литературы	31
Приложение 1. Текст программы с комментариями	33
Приложение 2. Снимки окон программы	44

Аннотация

В курсовой работе разработано мобильное приложение «Гостиница» для Android на языке Kotlin. Приложение реализует каталог гостиниц и номеров, поиск по параметрам проживания, бронирование нескольких физических комнат, выбор дополнительных услуг, личный кабинет и историю заказов. Администратор добавляет, редактирует и архивирует данные посредством интерфейса приложения. Данные хранятся в SQL-базе SQLite, доступ к которой организован библиотекой Room. Ранее разработанный сервер Ktor с PostgreSQL сохранён как отдельная реализация сетевого варианта.

Итоговая версия 0.9.0 предназначена для автономной учебной демонстрации: каталог, пользователи и заказы каждого устройства независимы. Первое открытие и основные операции не требуют интернета или работающего ноутбука. Встроенные карты Mapsforge содержат данные небольших центральных районов шести городов. Онлайн-оплата моделируется без списания денег. HTML-подтверждение можно сохранить и передать установленному почтовому клиенту; автоматическая доставка письма не заявляется. Код документирован русскими комментариями и KDoc, проект размещается в GitHub. Проверки включают модульные, интеграционные SQLite и автоматизированные интерфейсные тесты.

Ключевые слова: Android, Kotlin, гостиница, бронирование, SQL, Room, MVVM, offline-карта, демонстрационная оплата.

Abstract

This coursework presents Hotel, an Android application developed in Kotlin. It provides a structured hotel and room catalogue, parameter-based search, multi-room bookings, optional services, user profiles, booking history and an administrator interface. The standalone version uses SQLite through Room. A previously developed Ktor and PostgreSQL backend is retained as a separate client-server implementation.

Version 0.9.0 is an autonomous educational demonstration. Its catalogue, accounts and orders are independent on each device; first launch and core workflows do not require internet access or a running laptop. Mapsforge renders bundled map extracts for central areas of six cities. Payments are simulated without charging money. HTML confirmations can be saved and shared with an installed email client; automatic email delivery is not provided. The project includes Russian code documentation, unit tests, SQLite integration tests and automated interface tests.

Keywords: Android, Kotlin, hotel booking, SQL, Room, MVVM, offline map, simulated payment.

Введение

Планирование поездки включает выбор направления, оценку условий проживания и сравнение стоимости размещения. Сведения о категории номера, вместимости, дополнительных услугах и сроках отмены должны быть представлены согласованно. Если наличие проверяется только на день заезда или цена рассчитывается исключительно экраном, пользователь может получить противоречивый результат. Поэтому задача разработки охватывает не только интерфейс каталога, но и правила хранения, проверки доступности и изменения состояния заказа.

Цель работы - разработать на Kotlin мобильное Android-приложение «Гостиница», соответствующее индивидуальному заданию Арзимуротова А.М., группа ПИН-123. Задание выдано 18 сентября 2026 года, срок сдачи - 11 декабря 2026 года. Требуются структурированный каталог номеров, фильтрация, бронирование, дополнительные услуги, личный кабинет, история, SQL и кабинет администратора. Дополнительно заданы Dokka, GitHub, модульные проверки, функциональные испытания не менее чем на трёх виртуальных устройствах и автоматизированное тестирование интерфейса.

Для достижения цели анализируются предметная область и существующие решения, выделяются сущности и бизнес-правила, проектируется архитектура, реализуются сценарии пользователя и администратора, проверяются транзакции и интерфейс. Объект исследования - процессы выбора и резервирования гостиничного размещения. Предмет исследования - программные средства автоматизации этих процессов на мобильном устройстве.

Разработка выполнялась поэтапно. Этапы 1-6 сформировали интерфейс и серверную систему Android - Ktor - PostgreSQL. Для итоговой демонстрации на произвольном телефоне без внешней инфраструктуры выбран автономный режим 0.9.0. SQL-требование сохраняется: SQLite является реляционной SQL-СУБД, а Room - средством доступа к ней. Это изменение области эксплуатации, а не имитация сохранения в оперативной памяти. Сетевой backend не удалён, но не используется самостоятельно установленным APK.

1. Анализ технического задания

1.1. Предметная область и границы системы

Каталог объединяет несколько государств и городов. Страна определяет валюту, город - географическую принадлежность и часовой пояс, гостиница - название, адрес, звёздность и описание. Тип номера задаёт вместимость, площадь и удобства, физическая комната определяет конкретный ресурс размещения. Такое разделение позволяет показать общие условия типа, но резервировать точно определённые комнаты.

Гостиницы имеют от трёх до пяти звёзд. Начальный учебный каталог включает Узбекистан, Россию и Турцию; Ташкент, Самарканд, Москву, Санкт-Петербург, Стамбул и Анталию. В каждом городе шесть вымышленных гостиниц, в каждой три типа номеров и четыре физических комнаты каждого типа. Всего предусмотрены 36 гостиниц, 108 типов и 432 комнаты. Пять категорий городских мест образуют 30 информационных объектов. Названия, адреса, оценки и координаты этих объектов демонстрационные; приложение не выдаёт их за реальные коммерческие предложения.

Заказ содержит владельца, даты проживания, число взрослых и детей, выбранный тип, выделенные комнаты, услуги, сумму и валюту. В итоговой системе один заказ включает от одной до четырёх комнат одного типа. Даты рассматриваются как полуоткрытый интервал: день выезда не занимает ночь следующего гостя. Тариф должен покрывать весь интервал. Каждая выбранная физическая комната свободна в течение всей поездки, включая периоды административного закрытия.

Дополнительные услуги включают завтрак, встречу в аэропорту и другие разовые опции. Услуги с оплатой за ночь и комнату умножаются на обе величины; разовые оплачиваются один раз на заказ. Для трансфера сохраняются аэропорт, рейс, местное время встречи и телефон. Эти сведения показывают структуру заказа, но не направляются настоящему перевозчику.

Рестораны, кафе, парки, музеи и места отдыха связаны с городом. Отдельная связь с гостиницей исключает дублирование ресторана для каждой гостиницы. В этой версии места можно изучать и открывать на карте; билеты, столики, такси и автомобильная аренда не продаются. Они рассматриваются

как возможное дальнейшее развитие, а не незавершённые кнопки основного приложения.

Самостоятельный АРК является независимой учебной системой каждого телефона. Он не проверяет общую доступность у реальной гостиницы, не синхронизирует заказы между устройствами и не проводит банковские операции. Работа с включённым интернетом не меняет локальный режим. Такое ограничение обеспечивает воспроизводимую защиту проекта без сервера, но должно явно сообщаться пользователю.

1.2. Роли и пользовательские сценарии

Гость просматривает каталог, задаёт направление, даты и гостей, применяет фильтры, открывает галерею, описание номеров и городские места. Регистрация добавляет локальный аккаунт USER; роль ADMIN через обычную форму получить нельзя. Зарегистрированный клиент оформляет собственные заказы, выполняет демооплату, редактирует имя и телефон, меняет пароль, просматривает историю и разрешённые отмены.

Администратор управляет справочниками, комнатами, тарифами, услугами, фотографиями и рекомендациями гостиниц, закрывает комнаты на период и просматривает заказы. Отмена заказа администратором подчиняется той же сохранённой политике срока. Записи архивируются, а не удаляются вместе с историей. Каждый вызов административного бизнес-сервиса проверяет роль; скрытие кнопки не считается достаточной авторизацией.

Основной сценарий состоит из выбора страны и города, будущих дат, числа гостей и комнат; поиска доступного предложения; выбора типа и услуг; проверки итоговой суммы; входа; создания заказа; выбора ONLINE_DEMO или PAY_AT_HOTEL; последующего управления заказом. После успешной демооплаты формируется HTML-документ со снимком реквизитов. Для оплаты в гостинице формируется только подтверждение неоплаченного бронирования.

1.3. Обзор аналогов

Booking.com представляет пример последовательного выбора размещения и управления поездкой. Официальная страница приложения описывает бронирование жилья, изменение заказов и чтение ключевой информации без интернета [1]. Для проекта полезна понятная последовательность действий, а не копирование оформления, названия или коммерческой базы.

Островок ориентирован на поиск и бронирование гостиниц. На официальном сайте представлены международный каталог и выбор оплаты заранее либо в гостинице; также описан offline-доступ к совершённым бронированиям [2]. Этот подход помогает разделить подтверждение заказа и факт оплаты. В учебном проекте предварительная оплата заменена демонстрационной операцией.

Tripadvisor объединяет информацию о гостиницах, ресторанах и достопримечательностях [3]. Для проекта существенна связь размещения с изучением города. Пользовательские оценки аналогов не следует смешивать с учебными рейтингами: в разработанной системе рейтинг вводится администратором, а публикация отзывов клиентами не реализована.

Google Maps служит примером представления объектов на карте и их изучения [4]. Однако итоговое приложение использует Mapsforge и заранее подготовленные OSM-фрагменты, поскольку обязательными свойствами стали отсутствие API-ключа и работа при первом запуске без сети. Карта не обеспечивает навигацию по дорогам и не заменяет глобальный картографический сервис.

Аналог	Принцип, использованный в проекте
Booking.com	Последовательность поиска, выбора размещения и управления заказом; сохранение ключевых сведений
Островок	Разделение способов оплаты заранее и в гостинице
Tripadvisor	Каталог городских мест рядом с размещением

Аналог	Принцип, использованный в проекте
Google Maps	Маркеры и географическая привязка; самостоятельная реализация через Mapsforge

Сравнение относится к открыто наблюдаемым пользовательским функциям. Закрытые административные системы аналогов и их внутренние SQL-схемы не исследовались; наличие конкретной административной операции у них не утверждается. Разработанное приложение уступает коммерческим сервисам в охвате, поддержке и реальных услугах, но позволяет воспроизводимо изучить весь локальный жизненный цикл заказа и SQL-управление каталогом.

1.4. Требования и критерии корректности

Функциональные требования включают поиск по направлению, датам, гостям и комнатам, цене, звёздам, рейтингу, типу, удобствам, расстоянию и названию; устойчивую сортировку; просмотр условий; атомарное бронирование; кабинет и административное редактирование. Список результатов выдаётся страницами по двадцать записей. Наличие является результатом проверки на конкретный момент и повторно проверяется при создании заказа.

Нефункциональные требования: русскоязычный интерфейс, отсутствие блокировки UI при SQL и хешировании, сохранение после перезапуска, понятные ошибки, защита паролей и сеанса, отсутствие секретов в Git и сохранение истории при редактировании каталога. Все деньги хранятся в минимальных единицах Long/INTEGER; вычисления с Float или Double не используются. Смешанные валюты не сравниваются через неявный курс.

Требование задания	Компонент итоговой реализации
Android, Kotlin	Android Studio, Compose, Material 3, Navigation, ViewModel
Каталог и условия проживания	SQL-справочники, типы, вместимость, площадь, удобства, тарифы, галерея

Требование задания	Компонент итоговой реализации
Фильтрация и бронь	LocalEngine, Room transactions, физические комнаты и интервалы
Дополнительные услуги	hotel_services и неизменяемые снимки в заказе
Личный кабинет и история	Локальные аккаунты, бронирования, платежи, отмены
SQL и кабинет администратора	Room/SQLite, LocalAdmin, формы и журнал изменений
Dokka и GitHub	Русские KDoc, Gradle Dokka, репозиторий и Windows workflow
Тестирование	JVM, SQLite instrumentation, Compose E2E; отдельный протокол AVD
Графические материалы	Архитектура, схема данных, последовательность заказа и снимки экранов

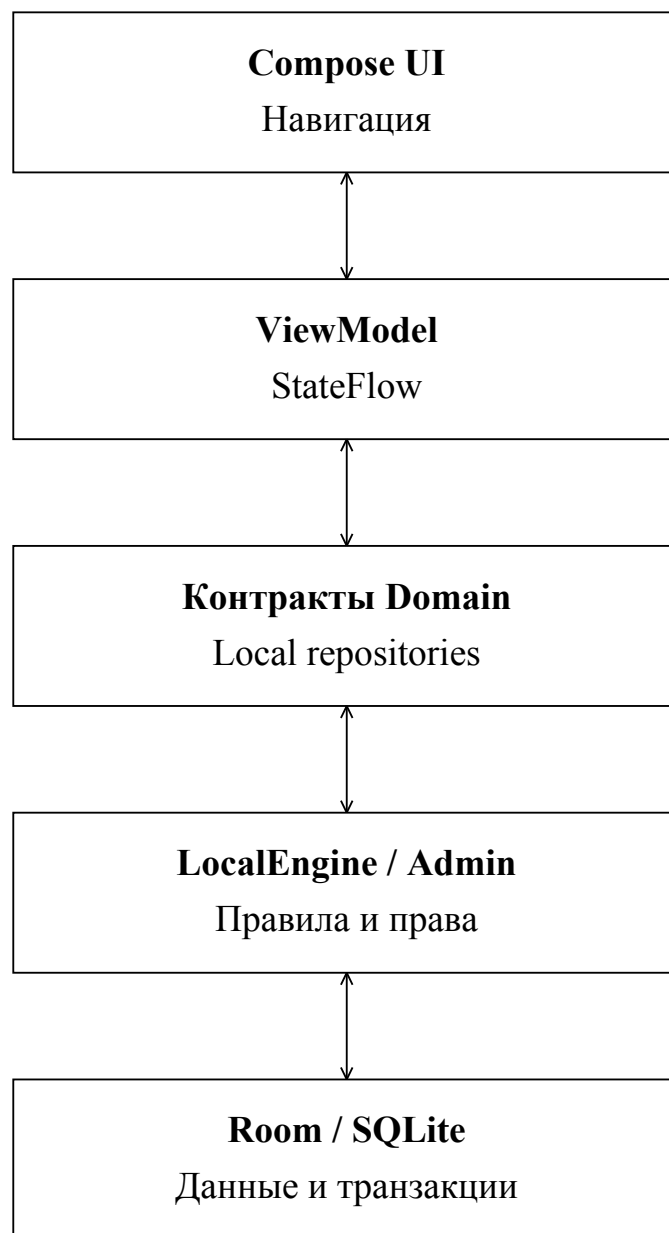
2. Разработка алгоритмов

2.1. Архитектура итоговой системы

Архитектура Android сохраняет разделение UI-Domain-Data и MVVM. Compose отображает состояние, ViewModel принимает события и публикует StateFlow, репозитории реализуют контракты предметной области. Hilt связывает рабочие контракты с LocalCatalogRepository, LocalAuthRepository и LocalTravelRepository. LocalEngine объединяет предметные правила и транзакции, а LocalDao выполняет SQL. Благодаря этому те же экраны не содержат запросов SQLite и не обращаются к сети напрямую.

Схема, выбранная на этапах 3-6, включала Android, Ktor REST API и PostgreSQL. Её сервер, маршруты, JWT, миграции V1-V5 и PDF-генератор сохраняются в каталоге server. В версии 0.9.0 рабочие Android-репозитории к нему не привязаны. Не следует описывать самостоятельный APK как HTTP-клиент с кэшем: Room здесь основной источник истины. Установка на другой телефон создаёт новую базу, а не подключает его к общей гостиничной системе.

Рисунок 2.1 - Компоненты самостоятельного APK



Mapsforge: локальные assets; сеть не используется.

Mapsforge отображает локальные бинарные map-файлы, подготовленные из OSM XML. Приложение не использует слой скачивания тайлов. Файл копируется из assets во внутреннее хранилище, затем читает MapFile. Нативный MapView размещён в Compose через AndroidView, а при выходе освобождает renderer и cache. Каталог и маркеры используют одинаковые идентификаторы.

Фоновые вычисления выполняются через Coroutines на Dispatchers.IO. Загрузка, ошибка, пустой результат и успешное состояние представлены явно. Независимые задачи поиска и карточки отменяются при новом запросе; поздний результат не заменяет более свежий. Операции заказа не выполняются одновременно одним ViewModel. При смене пользователя очищаются его экранные заказы и документ, но не SQL-история.

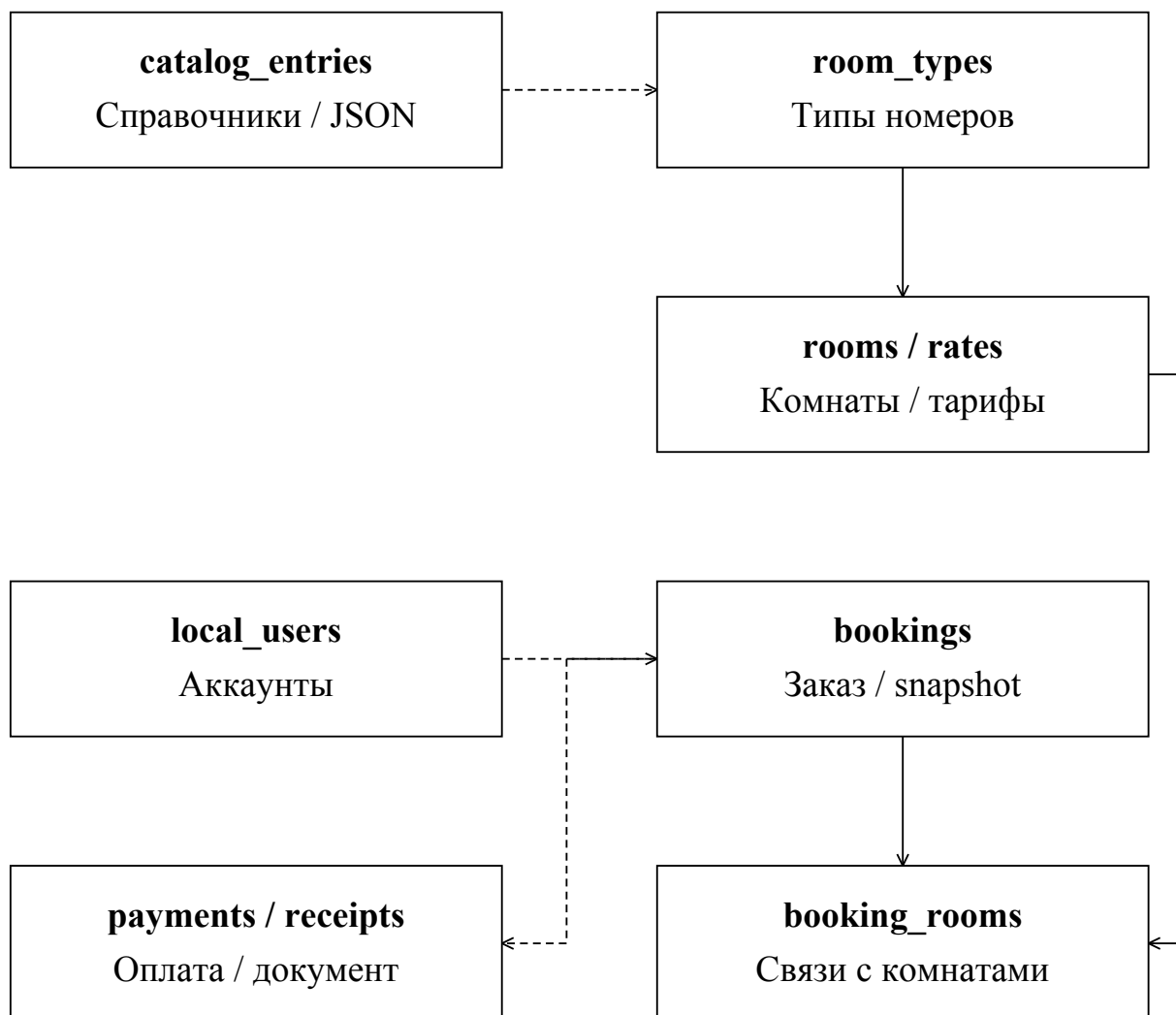
2.2. Структуры данных и SQL-схема

Основная база hotel_offline_09.db использует Room version 1 с экспортом схемы в Git. catalog_entries хранит отдельные записи стран, городов, гостиниц, мест и удобств: id, kind, title, payload, active, version. payload содержит JSON одного объекта, а не общий файл всего каталога. По kind создан индекс. Родительские связи внутри справочных JSON-объектов проверяет LocalAdmin; это не физические внешние ключи отдельных country/city таблиц. Такое проектное упрощение явно отличается от нормализованной серверной PostgreSQL-схемы.

Часто изменяемые остатки, интервалы, пользователи и заказы выделены в типизированные SQL-таблицы. room_types связывает тип с гостиницей и хранит вместимость, площадь, коды удобств и active. rooms содержит тип и номер комнаты; rates - тип, цену, валюту, границы и условия оплаты/отмены. Между rooms/rates и room_types, room_blocks и rooms, booking_rooms и bookings/rooms заданы внешние ключи с RESTRICT. Цены записываются INTEGER, а не REAL.

bookings хранит владельца, гостиницу, тип, даты, idempotency key, fingerprint и JSON-снимок заказа. booking_rooms задаёт выделение физических комнат. Сведения о выбранных услугах входят в неизменяемый BookingSnapshot; отдельной таблицы booking_services в локальной версии нет. Это не следует смешивать с сервером, где такая таблица существует. payments имеет уникальную связь с заказом; receipts хранит исходный оплаченный снимок для повторного HTML-формирования.

Рисунок 2.2 - Основные группы SQL-сущностей



Также: room_blocks, hotel_services, hotel_places, audit.

Пунктир: логическая связь; сплошная: SQL-связь.

Таблица SQLite	Назначение и ограничения
catalog_entries	Справочные записи; PK id, индекс kind, JSON одного объекта, active/version
local_users	Email UNIQUE, роль, Argon2id-хеши пароля/кода, версия сеанса

Таблица SQLite	Назначение и ограничения
room_types / rooms	Типы и физические ресурсы; номер уникален в гостинице по бизнес-правилу
rates / room_blocks	Цены и интервалы; пересечение тарифов/занятых блокировок отклоняется сервисом
hotel_services / hotel_places	Услуги гостиницы и связь с общими местами города
bookings / booking_rooms	Заказ, снимок и атомарное выделение комнат; UNIQUE ownerId/key
payments / receipts	Одна демооплата и один оплаченный снимок на заказ
audit	Локальный журнал действий ADMIN без паролей и восстановительных кодов

Начальный seed выполняется только для новой базы внутри транзакции. Детерминированные идентификаторы обеспечивают воспроизводимость. Маркер __seed09 не позволяет повторному запуску заменить отредактированные сведения или сбросить пароли. При обновлении APK база сохраняется; destructive migration не используется. Будущая смена структуры должна сопровождаться явной Room Migration и тестом перехода. Удаление данных пользователем средствами Android остаётся внешней операцией и уничтожает локальную демонстрацию.

2.3. Алгоритм поиска и доступности

Сначала проверяются существование направления, даты ISO, число гостей и комнат, допустимость фильтров и длина проживания. Начало не может быть в прошлом, конец должен быть позже начала, длительность - от одной до девяноста ночей. На каждую комнату нужен хотя бы один взрослый; необходимая вместимость определяется распределением суммарного числа гостей по одинаковым комнатам.

После фильтрации гостиниц выбираются активные типы, удовлетворяющие вместимости, категории и удобствам. Для типа нужен активный тариф в валюте страны, покрывающий весь интервал. Отдельные тарифы для разных частей поездки в этой версии не объединяются. Активные тарифы одного типа не могут пересекаться: правило проверяется при административной записи внутри общей транзакции.

Комната считается занятой при пересечении интервалов $start < checkOut$ и $end > checkIn$. Из доступного множества исключаются комнаты активных закрытых периодов, подтверждённых заказов и неистёкших `PENDING_PAYMENT`. Отменённый и истёкший резерв ресурс не занимает. Если свободных комнат меньше запрошенного числа, предложение не возвращается. Один физический ресурс должен быть свободен весь период; чередование разных комнат по ночам не используется.

Предложения сортируются устойчиво: при равной цене, рейтинге или расстоянии применяется идентификатор гостиницы. Пагинация берётся после общей сортировки, чтобы повторные страницы не меняли порядок. В локальной реализации DAO читает типизированные SQL-записи, а небольшой начальный каталог фильтруется бизнес-сервисом; сложность пропорциональна числу типов и интервалов. Для коммерческого объёма потребуется перенести фильтрацию в агрегирующие SQL-запросы и общий backend.

2.4. Атомарное создание заказа

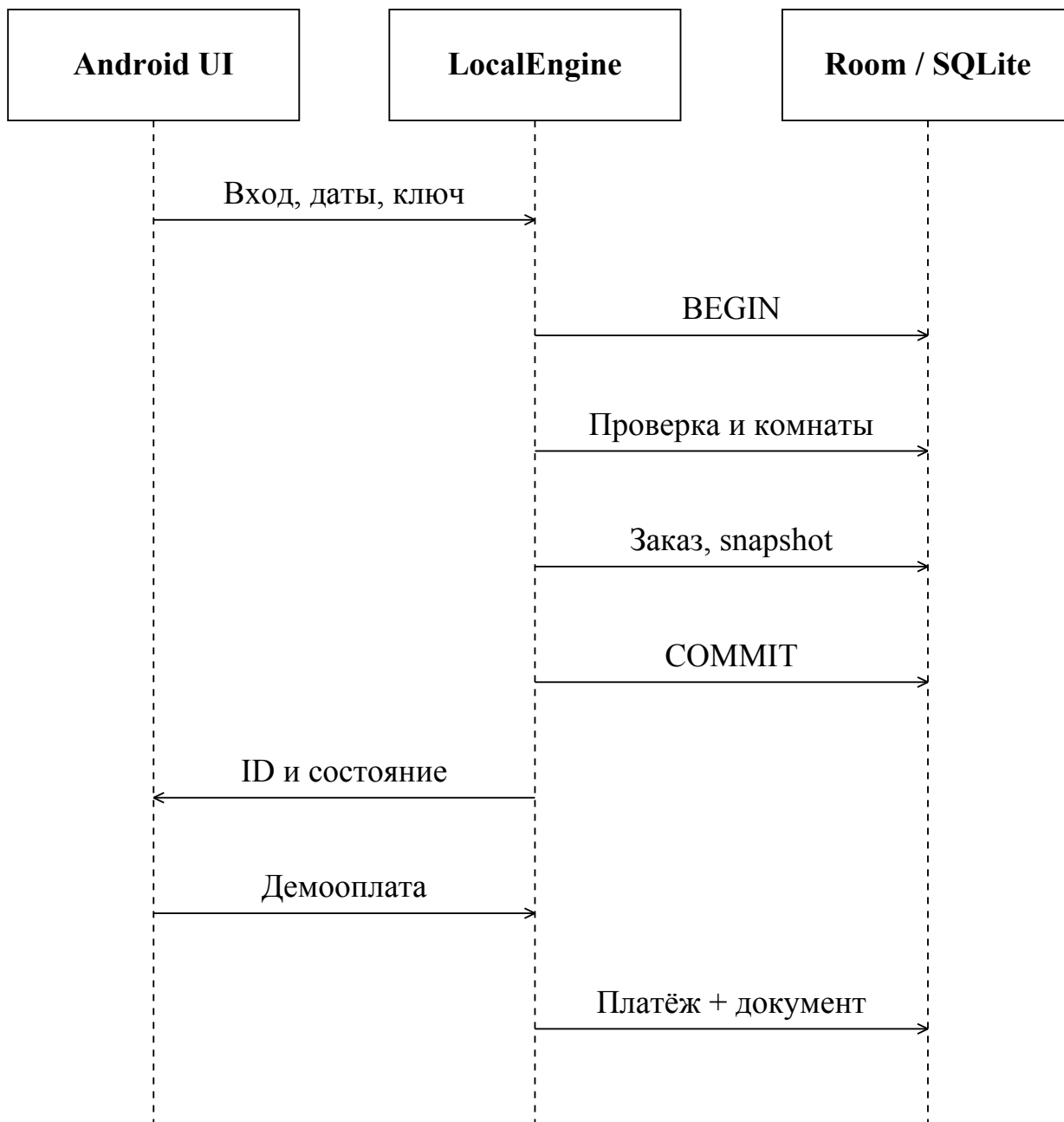
После входа проверяется, что указанный пользователь совпадает с текущим сеансом. Фиксируется `idempotency key` и SHA-256 отпечаток канонического запроса: множества услуг/удобств/звёзд сортируются, сам ключ не входит в отпечаток. Повтор одинакового запроса возвращает прежний заказ. Повтор ключа с другим содержимым отклоняется, поэтому двойное нажатие не создаёт второй заказ.

В одной `Room withTransaction` повторно проверяются действующий каталог, тариф, способ оплаты и количество комнат. Выделяются конкретные свободные ресурсы, рассчитываются проживание и услуги. Проживание равно ночной цене, умноженной на число ночей и комнат. Сумма услуг `PER_NIGHT_ROOM` также умножается на обе величины; `ONCE` добавляется

один раз. Операции используют проверяемые целочисленные умножения и сложения. Если сумма не совпадает с принятой клиентом, возвращается PRICE_CHANGED без создания заказа.

Все записи заказа и выделенных комнат фиксируются вместе. При недостаточном наличии возвращается AVAILABILITY_CHANGED, при исключении транзакция не оставляет частичного заказа. SQLite сериализует запись; параллельные операции не могут обе принять один устаревший остаток. Это механизм локальной базы, а не PostgreSQL row-level locks. В серверной реализации сохранён другой алгоритм блокировки физических комнат по UUID.

Рисунок 2.3 - Последовательность заказа и демооплаты



HTML читается из снимка после успешной транзакции.

Снимок фиксирует название и адрес гостиницы, клиента, комнатную категорию, номера, ночную цену, общую цену проживания, услуги с реквизитами трансфера и часовой пояс. Срок отмены вычисляется при создании и также сохраняется. Последующее редактирование гостиницы, профиля или тарифа не пересчитывает существующий заказ.

2.5. Состояния заказа и демооплаты

ONLINE_DEMO создаёт PENDING_PAYMENT/UNPAID с expiresAt через пятнадцать минут. PAY_AT_HOTEL создаёт CONFIRMED/UNPAID без резерва оплаты. При демооплате повторно проверяются владелец, статус, способ и время; payment, receipt и CONFIRMED/PAID сохраняются одной транзакцией. Одна успешная демооплата на заказ обеспечивается уникальным ограничением. Повтор успешной операции возвращает прежний transactionId и не продлевает резерв.

При достижении expiresAt резерв получает CANCELLED с причиной EXPIRED. Проверка остатков исключает истёкший резерв даже до обновления его текстового статуса. Во время чтения/записи заказов housekeeping обновляет истёкшие резервы и завершившиеся проживания. В автономной версии не обещается ежеминутная работа закрытого процесса Android: после открытия выполняется сверка по текущему времени устройства.

Заезд установлен на 14:00, выезд - 12:00 по IANA-зоне города. Подтверждённый заказ разрешено отменять до cancelUntil; pending-заказ отменяется до оплаты. Оплаченная демонстрационная операция становится REVERSED, но банковский возврат не проводится. После окончания проживания состояние меняется на COMPLETED. Изменение часов телефона может повлиять на эти сроки, что открыто описано в руководстве.

2.6. Аккаунты и восстановление доступа

Email нормализуется и уникален в локальной базе. Пароль хранится как индивидуально посоленный Argon2id-хеш: 19 МиБ памяти, два прохода, параллелизм один [5]. Хеширование выполняется вне UI-потока. Открытый пароль не пишется в базу, лог, SavedState или Git. Внутренний сеанс содержит пользователя и версию; он шифруется AES-GCM неэкспортируемым ключом Android Keystore [6]. Новое пространство noBackup-файлов не удаляет сохранённый серверный сеанс старой версии.

При первом входе создаётся случайный индивидуальный код восстановления. В Room сохраняется его Argon2id-хеш. До подтверждения получения зашифрованная копия хранится в noBackup, чтобы закрытие процесса не лишило пользователя кода. После подтверждения копия

удаляется. Восстановление требует email и код; успешное использование меняет пароль, повышает версию сеанса и выдаёт новый код. Смена пароля требует текущий пароль и тоже заменяет код.

Начальные учебные пароли известны для проверки проекта и не считаются секретами коммерческого сервиса. Они не дают доступа к чужому устройству: каждая база независима. Отсутствие кода и пароля не компенсируется скрытым универсальным входом. Взломанный/rooted телефон и подмена собственной SQLite-базы вне приложения находятся за границей модели угроз учебной демонстрации.

2.7. Администрирование, фотографии и карты

LocalAdmin проверяет роль перед каждой операцией. Формы используют метаданные AdminSchema, а связи выбираются из существующих активных объектов. Валюта тарифа должна совпадать со страной гостиницы; вместимость и координаты ограничены; пересекающиеся тарифы и закрытие занятой комнаты отклоняются. Версия записи предотвращает незаметное сохранение формы поверх более свежего изменения. Архивирование сохраняет идентификаторы и историю.

Фотография выбирается системным Photo Picker, копируется во внутреннее хранилище, проверяется по объёму и числу пикселей, уменьшается и перекодировается в JPEG без метаданных EXIF. SQL хранит только относительное случайное имя. Произвольный путь пользователя не принимается. Импортированные фото отображаются перед встроенными иллюстрациями; перестановка первого фото не требует повторного импорта. При неудачной записи новая копия удаляется, старые изображения не затираются.

Шесть map-файлов подготовлены Mapsforge map-writer из ограниченных OSM API /map выгрузок. Скрипт подготовки выполняется разработчиком, не телефоном; общедоступные tile-server не скачиваются массово. manifest.json содержит границы, URL, размер и SHA-256 каждого файла. На карте сохраняется атрибуция OpenStreetMap contributors и ODbL [7]. LGPL-лицензия библиотеки и ODbL карт относятся к соответствующим компонентам, а не объявляют весь каталог реальными OSM-объектами.

Отдельная запись администратора может находиться за пределами встроенного покрытия. Её список, описание и координаты доступны, но глобальная карта не подгружается скрыто. Расстояние вычисляется формулой гаверсинуса и означает прямую географическую дистанцию; маршрутизация и GPS-навигация не реализованы.

2.8. HTML-документ и передача почтовому клиенту

Документ формируется после успешной транзакции из оплаченного снимка. Его повторное открытие не создаёт платёж и не пересчитывает цену. В HTML включаются заказ, клиент, гостиница, номера, даты, услуги, сумма, валюта, способ и `demo transactionId`. Пользовательский текст экранируется. `WebView` отключает `JavaScript`, сеть и доступ к файловым `URI`; для отображения не загружаются внешние шрифты или изображения.

Системный `SAF` позволяет сохранить HTML без разрешения на всё хранилище. `FileProvider` выдаёт почтовому приложению временное `read`-разрешение на выбранное вложение. В `Gmail/Mail.ru` передаются адрес, тема, пояснение и HTML-файл; окончательную отправку выполняет пользователь. Наличие интернета не означает наличие настроенной почты, а открытие `chooser` не означает доставку. При отсутствии почтового клиента показывается объяснение и возможность сохранить файл. Автоматический `SMTP` и восстановление через `email` не входят в самостоятельную версию.

2.9. Подход к тестированию

Модульные тесты проверяют доменные ограничения, `ViewModel`, отмену устаревших запросов и защиту повторных действий. `Room instrumentation` тестирует настоящий `Android SQLite` в отдельной `in-memory` базе, не очищая производственные данные. `Fake Clock` позволяет проверить точную границу пятнадцати минут без ожидания. Параллельные корутины проверяют атомарное выделение нескольких комнат.

Интерфейсный сценарий через настоящую `MainActivity`, `Hilt`, `Room` и `Keystore` охватывает вход клиента, поиск, обе оплаты, HTML, карту, пересоздание `Activity` и административный экран. Отдельные `Compose`-тесты проверяют формы, фильтры, навигацию и состояния. Испытания запускаются

только на явно заданном emulator-serial; общий connectedDebugAndroidTest не используется, чтобы не затронуть личный телефон.

Три конфигурации функциональных испытаний, результаты запусков и ограничения оборудования фиксируются в docs/STAGES_7_9_VERIFICATION.md. Сборка, lint, Dokka, подпись и контрольная сумма APK проверяются отдельно: успешный unit-тест не доказывает работоспособность установки, а Windows CI не заменяет испытания на Android. Личные данные и секреты не публикуются в протоколе.

2.10. Выводы по главе

Разработанная архитектура отделяет Compose-интерфейс и состояние экранов от предметных правил и SQL-хранилища. Room обеспечивает атомарность заказа и оплаты, неизменяемые снимки сохраняют историю после редактирования каталога, а повторная проверка доступности предотвращает выделение одной комнаты пересекающимся заказам. Mapsforge и поставляемые карты позволяют исследовать центральные районы шести городов без сетевого сервиса.

Самостоятельная версия является локальной демонстрационной системой, а не распределённым гостиничным агрегатором. Ограничения часов устройства, размеров карты, ручной отправки писем и независимости баз явно отражены в интерфейсе и документации. Сохранённый серверный вариант не подменяет эти условия: для общего online-режима потребуется отдельная архитектура синхронизации и эксплуатации. Выбранное разделение компонентов обеспечивает основу для дальнейшего расширения и проверяемых сценариев пользователя и администратора.

3. Руководство программиста

3.1. Среда разработки и структура проекта

Для Android требуется Android Studio с SDK API 36, минимальная версия приложения - API 26. На Mac используется ARM64-эмулятор, на Windows Intel/AMD - соответствующий x86_64 образ. Проект открывается из android, но клонируется весь репозиторий: версии зависимостей находятся в общей gradle/libs.versions.toml, предметные DTO - в shared. Каталог server является независимым Gradle-проектом и не нужен для запуска самостоятельного APK.

android/app/src/main/java содержит launcher Activity, Compose-экраны и ViewModel в ui, Hilt-модуль в di, репозитории и защищённый сеанс в data, Room и LocalEngine/LocalAdmin в data/local. assets/maps содержит встроенные карты; drawable - исходные иллюстрации. app/schemas содержит экспорт Room. app/src/test и app/src/androidTest разделяют JVM и Android-проверки. docs хранит спецификации и протоколы, scripts - безопасные вспомогательные команды.

Версии Room, Mapsforge, Hilt, KSP, Ktor и Compose закреплены. Открытие в Android Studio требует первого Gradle Sync с доступом к Maven-зависимостям. После сборки установка APK и предметные операции не требуют этих серверов. На MacBook Air с 16 ГБ памяти эмуляторы запускаются последовательно, а не одновременно.

3.2. Сборка и запуск на Mac и Asus

Получите исходный код из <https://github.com/AbdullaArzimurotov/Hotel-Booking-Android.git>. В Android Studio выберите Open - android, дождитесь Sync и укажите встроенный JBR в Gradle JDK. Выберите конфигурацию app и устройство, нажмите Run. Для 0.9.0 не нужны adb reverse и локальный backend. Параметры SDK и абсолютные пути Mac не переносятся в Git; Windows создаёт свои local.properties и IDE-настройки.

В терминале Mac используйте ./gradlew :app:assembleDebug из android при настроенном JAVA_HOME. В PowerShell Asus выполняется .\gradlew.bat :app:assembleDebug. JVM-проверки запускаются :app:testDebugUnitTest,

статический анализ - :app:lintDebug, документация - :app:dokkaGenerate. scripts/ui_tests.py принимает обязательный serial эмулятора и отказывается от физического устройства.

Release-сборка использует приватные настройки .local/signing.properties и ключ .local/hotel-distribution.jks. Эти файлы не входят в GitHub. Distribution package ru.arzimurotov.hotel.offline отделён от ru.arzimurotov.hotel старой debug-сборки: можно сохранить обе установки без удаления данных. Их SQLite-базы независимы. Старую debug-установку обновляют её же подписанным debug APK через install -г. Нельзя заменять её другим сертификатом и для обхода ошибки удалять данные.

3.3. Изменение кода и SQL-схемы

Новые правила реализуются в LocalEngine/LocalAdmin, а не в обработчиках Compose. DAO получает минимальные операции SQL; запись взаимосвязанных объектов выполняется в transaction. Изменение публичной DTO должно сохранять обратную совместимость сохранённых JSON-снимков: новые поля имеют разумные defaults. Старые заказы не следует десериализовать в текущий изменяемый Hotel и пересчитывать заново.

При изменении таблиц повышается номер Room и добавляется неdestructивная Migration с тестом существующей базы. fallbackToDestructiveMigration не допускается. Применённые серверные Flyway-файлы также не редактируются: сетевой backend использует новую миграцию, но самостоятельная база автоматически с ним не синхронизируется. SQL-тесты backend выполняются только в отдельной тестовой базе.

Полезные точки расширения: собственный репозиторий общей сетевой системы, серверная рассылка с безопасным хранением SMTP-ключа, более широкие offline-карты и экспорт пользовательской резервной копии. Билеты, такси и аренда требуют отдельных сущностей и правил доступности. Их нельзя реализовать простой сменой подписи кнопки гостиничного заказа.

3.4. Документация и контроль версий

KDoc поясняет назначение компонентов, транзакционность, ограничения demo, источник данных и правила безопасности. Dokka собирает HTML-справочник из комментариев. Смысловые изменения сопровождаются русскими commit-сообщениями. В Git не попадают базы, пароль подписи, private key, сеанс, восстановительные коды или персональные документы. Начальные demo-пароли публичны намеренно и не совпадают с инфраструктурными секретами.

Mac и Asus выполняют git pull перед работой, а после проверки commit/push. Git переносит код, карты, схемы и документы, но не локальные заказы. Резервная копия ключа подписи хранится отдельно в зашифрованном виде. При утрате этого ключа существующую release-установку нельзя обновить другим сертификатом без нового приложения.

4. Руководство пользователя

4.1. Установка и первый запуск

Передайте signed APK на телефон и разрешите установку для выбранного источника средствами Android. После открытия создаётся локальная база с начальными объектами. Интернет можно отключить до первого открытия; ждать запуска ноутбука не требуется. При обновлении используйте тот же package и подпись. Не выполняйте uninstall или «Очистить данные», если нужно сохранить историю.

Главные разделы - «Поиск», «Места», «Брони» и «Профиль». На широком экране они отображаются боковой панелью, на телефоне - нижней навигацией. Диагностика standalone сообщает о SQLite, а не о подключении к Ktor. Начальные данные учебные; сообщение о demo присутствует в ключевых сценариях.

4.2. Вход, регистрация и пароль

Администратор входит как admin@hotel.demo с паролем HotelAdmin!2026; клиент - client@hotel.demo с HotelClient!2026. Это не настоящие ящики. Для своего аккаунта откройте регистрацию, введите имя, email, пароль и подтверждение, при желании телефон. Email должен быть уникален на этом устройстве. После регистрации войдите и сохраните индивидуальный восстановительный код.

В профиле редактируются имя и телефон. Для изменения пароля введите действующий пароль и дважды новый; сохраните новый код. Если пароль забыт, используйте email и последний сохранённый код в «Забыли пароль?». Без обоих способов доступа скрытого общего пароля нет. Для выхода нажмите «Выйти из аккаунта»; заказы остаются в SQLite и станут доступны при следующем входе этого пользователя.

4.3. Поиск и оформление проживания

Выберите страну/город, даты и гостей. Фильтры ограничивают цену в валюте страны, звёзды, рейтинг, удобства, тип и расстояние. Откройте карточку, ознакомьтесь с адресом, описанием, фотографиями и вместимостью. Выберите подходящий тип и услуги. Для трансфера время задаётся в формате ГГГГ-ММ-ДДТЧЧ:ММ по местной зоне выбранного города.

Перед подтверждением проверьте число комнат, услуги, сумму и способ оплаты. При изменении условий система отклонит устаревший расчёт и предложит обновить его. ONLINE_DEMO требует отдельного действия в кабинете до окончания пятнадцатиминутного резерва. PAY_AT_HOTEL сразу подтверждает заказ, но не показывает оплаченный чек. Банковские реквизиты не запрашиваются, деньги не списываются.

4.4. Кабинет, подтверждение и почта

В «Брони» переключайтесь между активными заказами и историей, просматривайте комнаты, услуги, сумму, время резерва и оплату. Пока отмена разрешена тарифом, нажмите «Отменить бронирование» и подтвердите действие. При изменении часов телефона сроки demo могут измениться; для корректной демонстрации оставляйте автоматическую дату и время Android.

После демооплаты откройте подтверждение. Оно хранит первоначальные реквизиты даже после позднейшей отмены; текущий статус оплаты берётся из кабинета. Нажмите «Сохранить HTML» или «Отправить на почту». Во втором случае откроется установленный почтовый клиент с вложением. Если использован demo-логин, замените получателя реальным email. Самостоятельно отправьте письмо при наличии интернета; приложение не выдаёт подготовку за доставку.

4.5. Места, карта и администрирование

В «Места» выбираются город и категория. Offline-карта отображает небольшие встроенные районы; нажмите маркер, затем «Открыть». Если координаты вне района, используйте описание и список. Связанные с гостиницей места показываются отдельно; расстояние по прямой не означает маршрут. Билеты, столики и транспорт в этой версии не заказываются.

Администратор открывает кабинет из профиля и выбирает вид данных. Родители выбираются из списков, даты задаются ISO, цены - в минимальных единицах валюты. Сохранение проверяет ссылки, диапазоны, тарифы и конфликты занятости. Неактивная запись архивируется. Фото можно добавить из галереи, поставить первым или убрать. Просмотр журнала позволяет установить, какой локальный администратор изменял данные.

Заключение

В ходе курсовой работы разработаны интерфейс Android на Kotlin, предметные алгоритмы доступности, атомарного заказа и демооплаты, SQL-модель, личный кабинет и административное управление. Версия 0.9.0 обеспечивает автономную проверку без ноутбука и интернета; Room является основным хранилищем, Mapsforge использует поставляемые карты, HTML-документ передаётся почтовому клиенту с явным разграничением подготовки и доставки.

Важными решениями стали разделение типа и физической комнаты, использование полуоткрытых интервалов, проверка всего проживания, целочисленные денежные расчёты, идемпотентность и неизменяемые снимки истории. Безопасность локального аккаунта поддерживается Argon2id, Keystore и индивидуальным кодом восстановления. Применены MVVM, StateFlow, Hilt и разделение интерфейса от SQL-правил.

Ограничения результата связаны с учебной областью эксплуатации: независимые базы устройств, доверие часам телефона, небольшой объём карты, отсутствие реальных платежей и автоматической рассылки. Сохранённый Ktor/PostgreSQL backend может стать основой будущей общей online-системы, но подключение к ней потребует отдельного решения синхронизации, авторизации, инфраструктуры и проверки данных. Билеты, такси и аренда остаются дальнейшим развитием за пределами индивидуального гостиничного задания.

Список используемой литературы

1. Booking.com. Mobile application.
URL: <https://www.booking.com/apps.html>
(дата обращения: 05.10.2026).
2. Островок. Бронирование гостиниц и способы оплаты.
URL: <https://ostrovok.ru/>
(дата обращения: 05.10.2026).
3. Tripadvisor. Hotels, restaurants and attractions.
URL: <https://www.tripadvisor.com/>
(дата обращения: 05.10.2026).
4. Google Maps Help. Find nearby places.
URL: <https://support.google.com/maps/answer/144349>
(дата обращения: 05.10.2026).
5. OWASP. Password Storage Cheat Sheet.
URL: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
(дата обращения: 05.10.2026).
6. Android Developers. Android Keystore.
URL: <https://developer.android.com/privacy-and-security/keystore>
(дата обращения: 05.10.2026).
7. OpenStreetMap. Copyright and License.
URL: <https://www.openstreetmap.org/copyright>
(дата обращения: 05.10.2026).
8. Android Developers. Room and asynchronous queries.
URL: <https://developer.android.com/training/data-storage/room/async-queries>
(дата обращения: 05.10.2026).
9. Mapsforge. Integration guide.
URL: <https://github.com/mapsforge/mapsforge/blob/master/docs/Integration.md>
(дата обращения: 05.10.2026).

10. Kotlin. Documentation tool Dokka.

URL: <https://kotlinlang.org/docs/dokka-introduction.html>

(дата обращения: 05.10.2026).

11. Семакова А. Введение в разработку приложений для смартфонов на ОС Android. М.: ИНТУИТ, 2016.

12. Гриффитс Д., Гриффитс Д. Head First. Программирование для Android. 2-е изд. СПб.: Питер, 2018. 912 с.

Приложение 1. Текст программы с комментариями

Ниже приведены ключевые фрагменты действующей реализации. Полный исходный текст, включая backend и тесты, находится в репозитории GitHub и поставляемой папке проекта. Фрагменты не являются отдельной сокращённой программой, а иллюстрируют транзакции, правила доступа и SQL-связи.

```
android/app/src/main/java/ru/arzimurotov/hotel/data/local/LocalDatabase.kt
```

```
package ru.arzimurotov.hotel.data.local
```

```
import androidx.room.*
```

```
/**
```

```
 * Справочные объекты сериализуются по одному, а не одним blob всей базы.  
 Индекс kind обеспечивает
```

```
 * разделение стран/городов/гостиниц/мест; active сохраняет архивные записи  
 . Остатки, интервалы,
```

```
 * пользователи и заказы вынесены в отдельные SQL-таблицы ниже.
```

```
 */
```

```
@Entity(tableName = "catalog_entries", indices = [Index("kind")])
```

```
data class CatalogEntry(  
    @PrimaryKey val id: String,
```

```
    val kind: String,
```

```
    val title: String,
```

```
    val payload: String,
```

```
    val active: Boolean = true,
```

```
    val version: Int = 1,
```

```
)
```

```
@Entity(tableName = "local_users", indices = [Index(value = ["email"], unique = true)])
```

```
data class LocalUser(  
    @PrimaryKey val id: String,
```

```
    val email: String,
```

```
    val name: String,
```

```
    val phone: String?,
```

```

    val role: String,
    val passwordHash: String,
    val recoveryHash: String? = null,
    val sessionVersion: Int = 1,
    val active: Boolean = true,
)

@Entity(tableName = "room_types", indices = [Index(value = ["hotelId", "kind"],
unique = true)])
data class LocalType(
    @PrimaryKey val id: String,
    val hotelId: String,
    val kind: String,
    val capacity: Int,
    val area: Int,
    val amenities: String,
    val active: Boolean = true,
)

@Entity(
    tableName = "rooms",
    indices = [Index(value = ["typeId", "number"], unique = true)],
    foreignKeys =
        [
            ForeignKey(
                entity = LocalType::class,
                parentColumns = ["id"],
                childColumns = ["typeId"],
                onDelete = ForeignKey.RESTRICT,
            )
        ],
)
data class LocalRoom(
    @PrimaryKey val id: String,
    val typeId: String,

```

```

    val number: String,
    val active: Boolean = true,
)

@Entity(
    tableName = "rates",
    indices = [Index("typeId")],
    foreignKeys =
        [
            ForeignKey(
                entity = LocalType::class,
                parentColumns = ["id"],
                childColumns = ["typeId"],
                onDelete = ForeignKey.RESTRICT,
            )
        ],
)

data class LocalRate(
    @PrimaryKey val id: String,
    val typeId: String,
    val amount: Long,
    val currency: String,
    val start: String,
    val end: String,
    val demo: Boolean = true,
    val atHotel: Boolean = true,
    val freeCancelHours: Int = 24,
    val active: Boolean = true,
)

@Entity(tableName = "hotel_services", indices = [Index("hotelId")])
data class LocalService(
    @PrimaryKey val id: String,
    val hotelId: String,
    val code: String,

```

```

    val name: String,
    val description: String,
    val amount: Long,
    val perNight: Boolean,
    val active: Boolean = true,
)

@Entity(
    tableName = "room_blocks",
    indices = [Index("roomId")],
    foreignKeys =
        [
            ForeignKey(
                entity = LocalRoom::class,
                parentColumns = ["id"],
                childColumns = ["roomId"],
                onDelete = ForeignKey.RESTRICT,
            )
        ],
)

data class LocalBlock(
    @PrimaryKey val id: String,
    val roomId: String,
    val start: String,
    val end: String,
    val reason: String,
    val active: Boolean = true,
)

@Entity(
    tableName = "bookings",
    indices = [Index(value = ["ownerId", "key"], unique = true), Index("hotelId")],
)

data class LocalOrder(
    @PrimaryKey val id: String,

```

```

        val ownerId: String,
        val hotelId: String,
        val typeId: String,
        val start: String,
        val end: String,
        val key: String,
        val fingerprint: String,
        val payload: String,
    )

@Entity(
    tableName = "booking_rooms",
    primaryKeys = ["bookingId", "roomId"],
    indices = [Index("roomId")],
    foreignKeys =
        [
            ForeignKey(
                entity = LocalOrder::class,
                parentColumns = ["id"],
                childColumns = ["bookingId"],
                onDelete = ForeignKey.RESTRICT,
            ),
            ForeignKey(
                entity = LocalRoom::class,
                parentColumns = ["id"],
                childColumns = ["roomId"],
                onDelete = ForeignKey.RESTRICT,
            ),
        ],
)
data class LocalAllocation(val bookingId: String, val roomId: String)

@Entity(
    tableName = "payments",
    indices =

```

```

        [
            Index(value = ["bookingId"], unique = true),
            Index(value = ["ownerId", "key"], unique = true),
        ],
    )
data class LocalPayment(
    @PrimaryKey val id: String,
    val bookingId: String,
    val ownerId: String,
    val key: String,
    val transactionId: String,
    val amount: Long,
    val currency: String,
    val paidAt: Long,
    val status: String = "PAID",
)

@Entity(tableName = "receipts", indices = [Index(value = ["bookingId"], unique = true)])
data class LocalReceipt(
    @PrimaryKey val id: String,
    val bookingId: String,
    val ownerId: String,
    val payload: String,
)

@Entity(
    tableName = "hotel_places",
    primaryKeys = ["hotelId", "placeId"],
    indices = [Index("placeId")],
)
data class LocalHotelPlace(val hotelId: String, val placeId: String)

@Entity(tableName = "audit")
data class LocalAudit(
    @PrimaryKey val id: String,

```

```

    val actor: String,
    val action: String,
    val target: String,
    val createdAt: Long,
)

/** DAO вызывается на Ю либо внутри withTransaction. Денежные суммы IN
TEGER/Long: без REAL. */
@Dao
interface LocalDao {
    @Query("SELECT * FROM catalog_entries ORDER BY title,id")
    suspend fun entries(): List<CatalogEntry>

    @Query("SELECT * FROM catalog_entries WHERE id=:id")
    suspend fun entry(id: String): CatalogEntry?

    @Upsert suspend fun put(v: CatalogEntry)

    @Query("SELECT * FROM local_users WHERE email=:email")
    suspend fun userEmail(email: String): LocalUser?

    @Query("SELECT * FROM local_users WHERE id=:id") suspend fun user(id:
String): LocalUser?

    @Query("SELECT count(*) FROM local_users") suspend fun userCount(): Int

    @Upsert suspend fun put(v: LocalUser)

    @Query("SELECT * FROM room_types ORDER BY id") suspend fun types():
List<LocalType>

    @Upsert suspend fun put(v: LocalType)

    @Query("SELECT * FROM rooms ORDER BY id") suspend fun rooms(): List
<LocalRoom>

    @Upsert suspend fun put(v: LocalRoom)

    @Query("SELECT * FROM rates ORDER BY start,id") suspend fun rates(): Li
st<LocalRate>

    @Upsert suspend fun put(v: LocalRate)

```

@Query("SELECT * FROM hotel_services ORDER BY id") suspend fun services(): List<LocalService>

@Upsert suspend fun put(v: LocalService)

@Query("SELECT * FROM room_blocks ORDER BY start,id") suspend fun blocks(): List<LocalBlock>

@Upsert suspend fun put(v: LocalBlock)

@Query("SELECT * FROM bookings ORDER BY start,id") suspend fun orders(): List<LocalOrder>

@Query("SELECT * FROM bookings WHERE id=:id") suspend fun order(id: String): LocalOrder?

@Upsert suspend fun put(v: LocalOrder)

@Query("SELECT * FROM booking_rooms") suspend fun allocations(): List<LocalAllocation>

@Insert(onConflict = OnConflictStrategy.IGNORE) suspend fun put(v: LocalAllocation)

@Query("SELECT * FROM payments WHERE bookingId=:id") suspend fun payment(id: String): LocalPayment?

@Upsert suspend fun put(v: LocalPayment)

@Query("SELECT * FROM receipts WHERE id=:id") suspend fun receipt(id: String): LocalReceipt?

@Upsert suspend fun put(v: LocalReceipt)

@Query("SELECT * FROM hotel_places") suspend fun links(): List<LocalHotelPlace>

@Insert(onConflict = OnConflictStrategy.IGNORE) suspend fun put(v: LocalHotelPlace)

@Query("DELETE FROM hotel_places WHERE hotelId=:id") suspend fun clearLinks(id: String)

@Insert suspend fun put(v: LocalAudit)

```

    @Query("SELECT * FROM audit ORDER BY createdAt DESC LIMIT 100")
    suspend fun audit(): List<LocalAudit>
}

/**
 * Новая независимая БД: старые файлы сеанса 0.6.0 и сервер PostgreSQL не
 удаляются. Разрушительные
 * миграции запрещены; схемы Room экспортируются в Git для последующи
 х версий.
 */
@Database(
    entities =
        [
            CatalogEntry::class,
            LocalUser::class,
            LocalType::class,
            LocalRoom::class,
            LocalRate::class,
            LocalService::class,
            LocalBlock::class,
            LocalOrder::class,
            LocalAllocation::class,
            LocalPayment::class,
            LocalReceipt::class,
            LocalHotelPlace::class,
            LocalAudit::class,
        ],
    version = 1,
    exportSchema = true,
)
abstract class LocalDatabase : RoomDatabase() {
    abstract fun dao(): LocalDao
}

android/app/src/main/java/ru/arzimurotov/hotel/data/local/LocalSecurity.kt

```

```

package ru.arzimurotov.hotel.data.local

import java.security.MessageDigest
import java.security.SecureRandom
import java.util.Base64
import org.bouncycastle.crypto.generators.Argon2BytesGenerator
import org.bouncycastle.crypto.params.Argon2Parameters

/**
 * Argon2id: 19 MiB, 2 прохода, p=1; отдельная случайная salt. Вызывается то
лько на IO.
 * Восстановительный код тоже хешируется. Не является авторизацией общег
о облачного сервера.
 */
object LocalSecurity {
    private val random = SecureRandom()

    fun hash(value: String): String {
        val salt = ByteArray(16).also(random::nextBytes)
        return Base64.getEncoder().encodeToString(salt) +
            ":" +
            Base64.getEncoder().encodeToString(derive(value, salt))
    }

    fun matches(value: String, stored: String): Boolean = runCatching {
        val parts = stored.split(':')
        val salt = Base64.getDecoder().decode(parts[0])
        val expected = Base64.getDecoder().decode(parts[1])
        salt.size == 16 &&
            expected.size == 32 &&
            MessageDigest.isEqual(expected, derive(value, salt))
    }
        .getOrElse(false)

    private fun derive(value: String, salt: ByteArray): ByteArray {
        val generator = Argon2BytesGenerator()
        generator.init(

```

```

        Argon2Parameters.Builder(Argon2Parameters.ARGON2_id)
            .withSalt(salt)
            .withMemoryAsKB(19456)
            .withIterations(2)
            .withParallelism(1)
            .build()
    )
    return ByteArray(32).also { generator.generateBytes(value.toByteArray(Char
sets.UTF_8), it) }
}

fun recoveryCode() =
    ByteArray(16)
        .also(random::nextBytes)
        .joinToString("") { "%02X".format(it) }
        .chunked(4)
        .joinToString("-")
}

```

Приложение 2. Снимки окон программы

Снимки получены из действующей программы на эмуляторе. Они демонстрируют интерфейс, а не дизайнерские макеты. Указанные учётные записи и заказы учебные.

Рисунок П2.1 - Поиск проживания

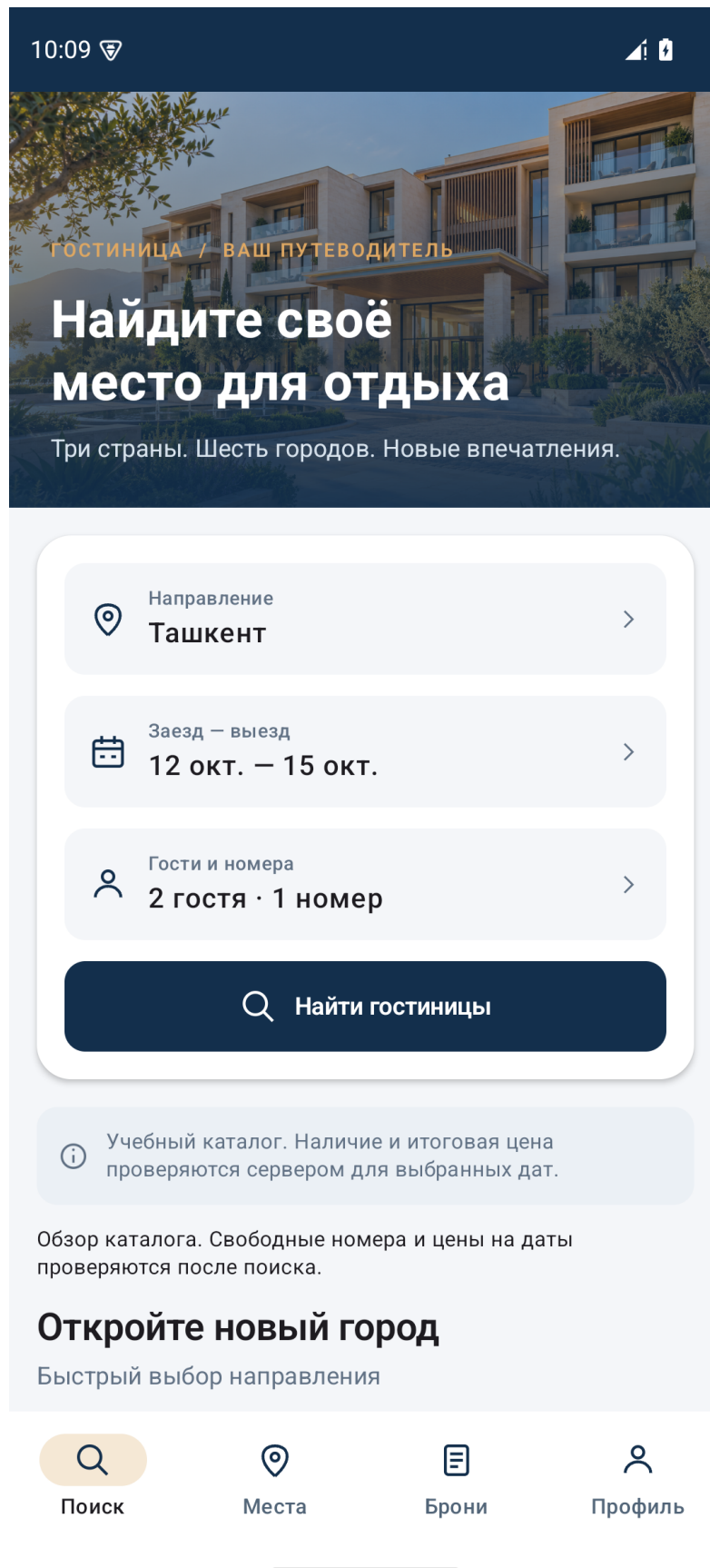


Рисунок П2.2 - Личный кабинет

10:09

Профиль

Путешествуйте в своём ритме

Демо-клиент

client@hotel.demo

Пользователь

Имя

Демо-клиент

Телефон

Сохранить профиль

Выйти из аккаунта

Изменить пароль

Приветственное письмо

Вы вошли в аккаунт.

Обновить состояние аккаунта

Поиск

Места

Брони

Профиль

Рисунок П2.3 - Заказ и демооплата

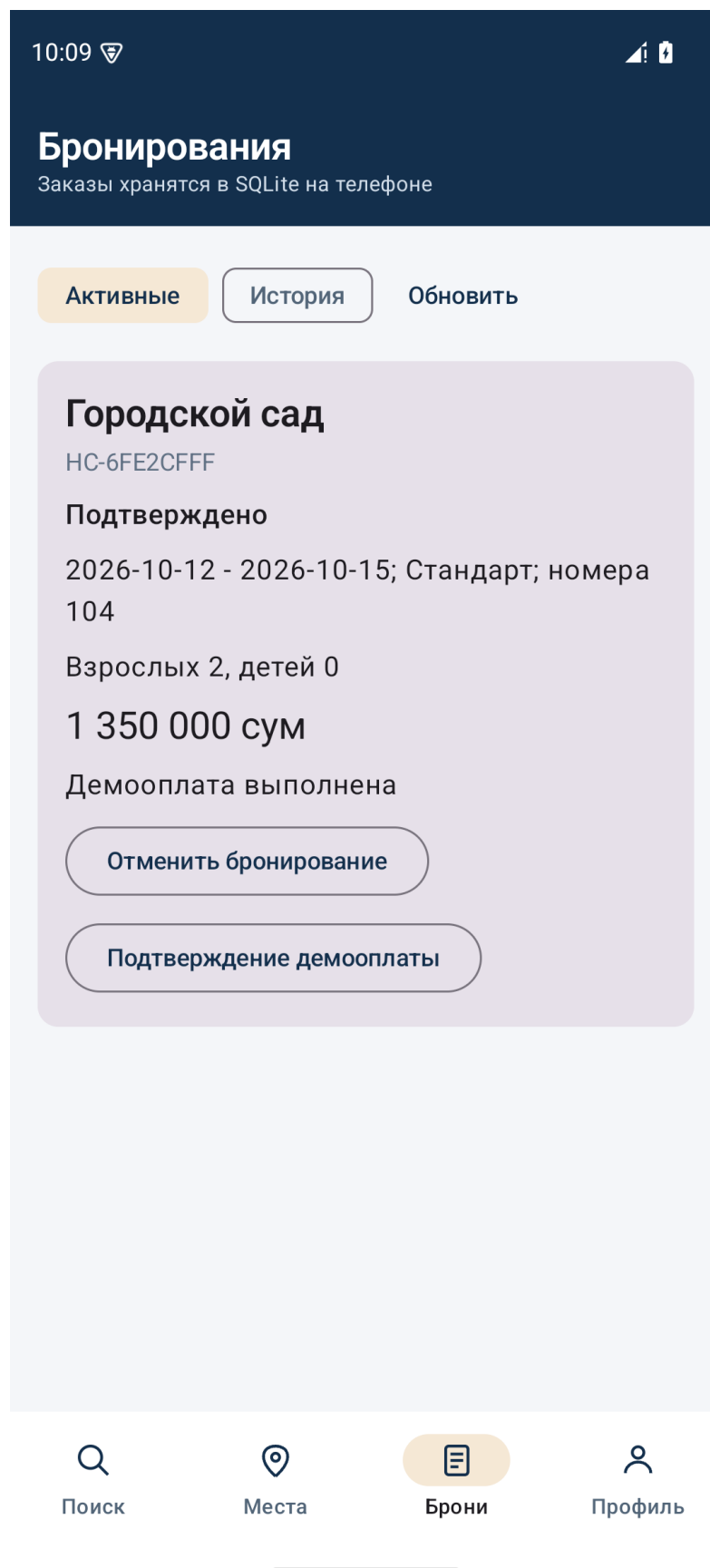


Рисунок П2.4 - HTML-подтверждение

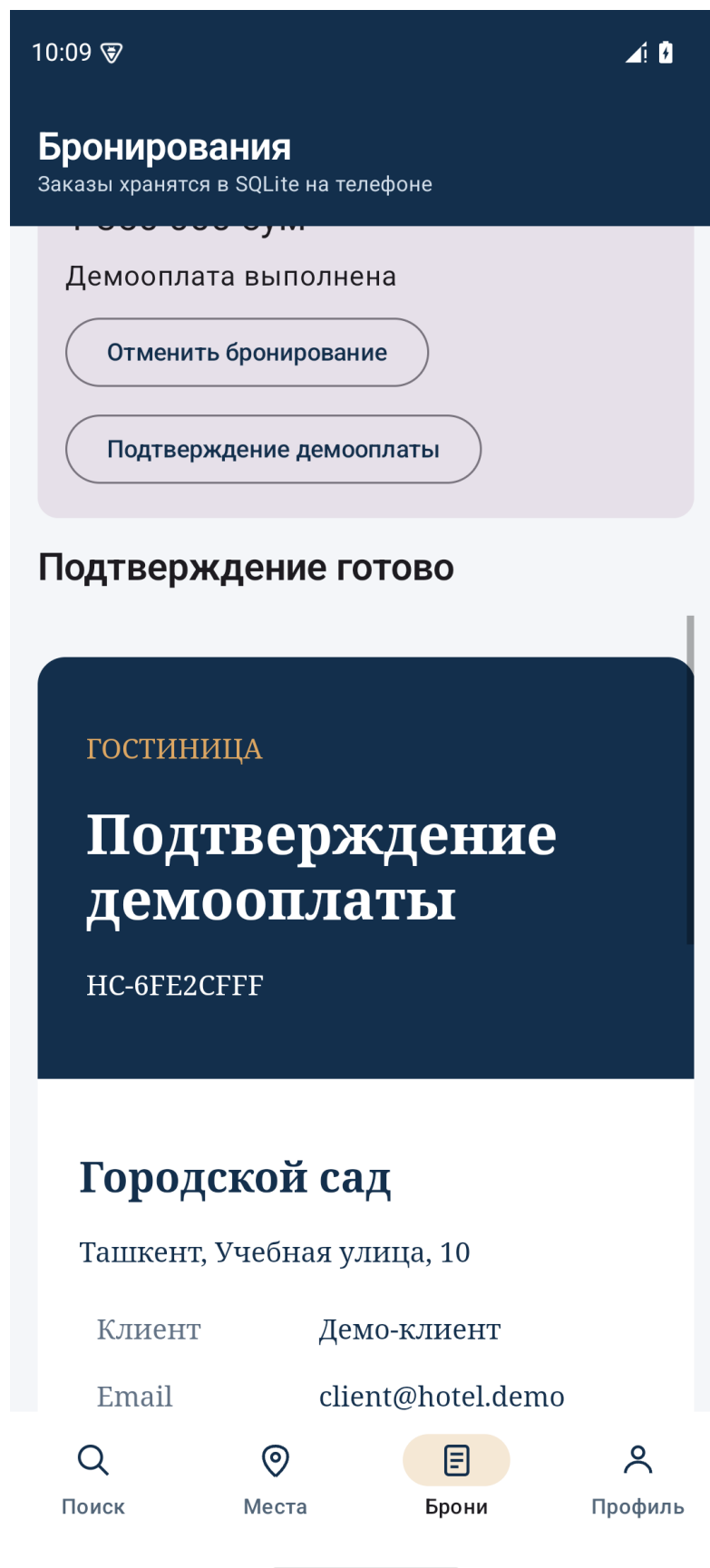


Рисунок П2.5 - Offline-карта

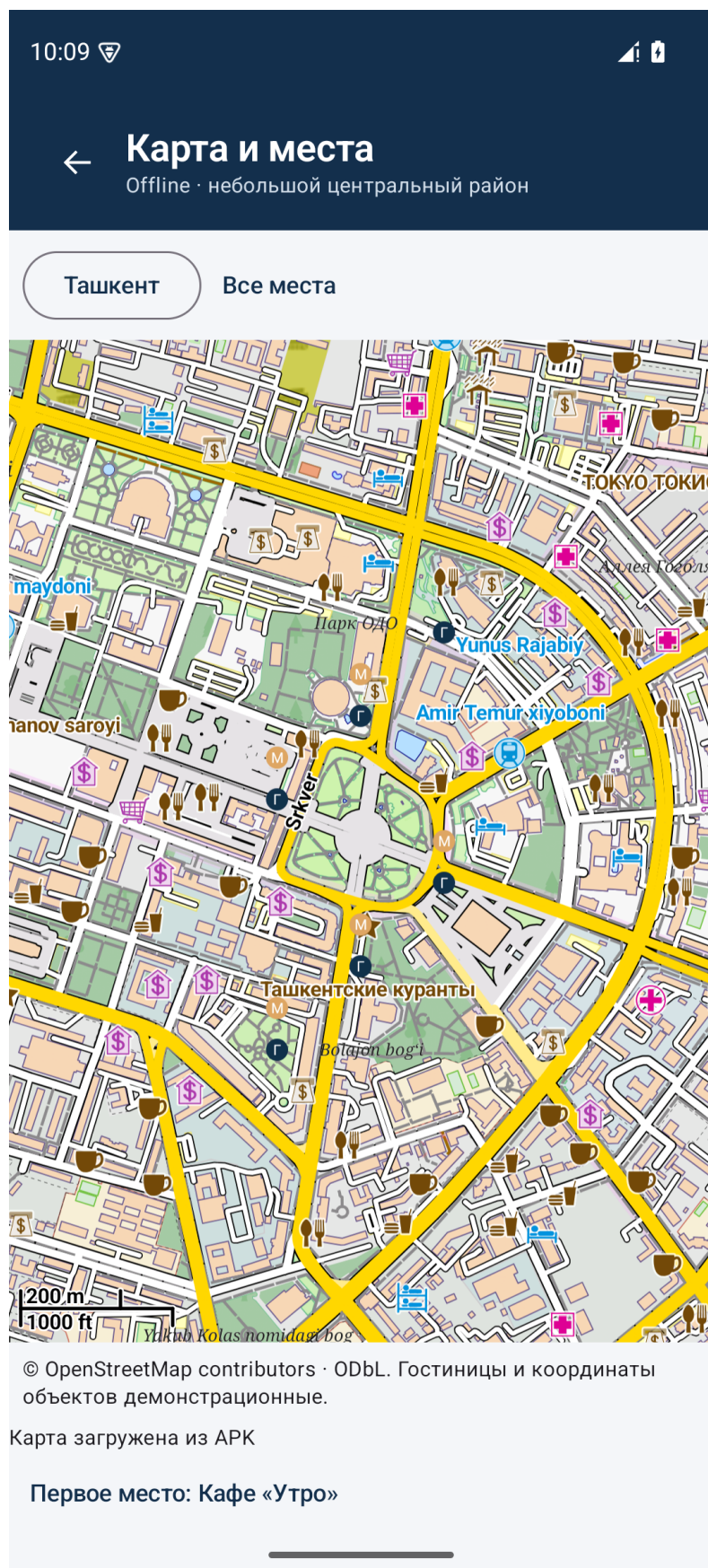


Рисунок П2.6 - Кабинет администратора

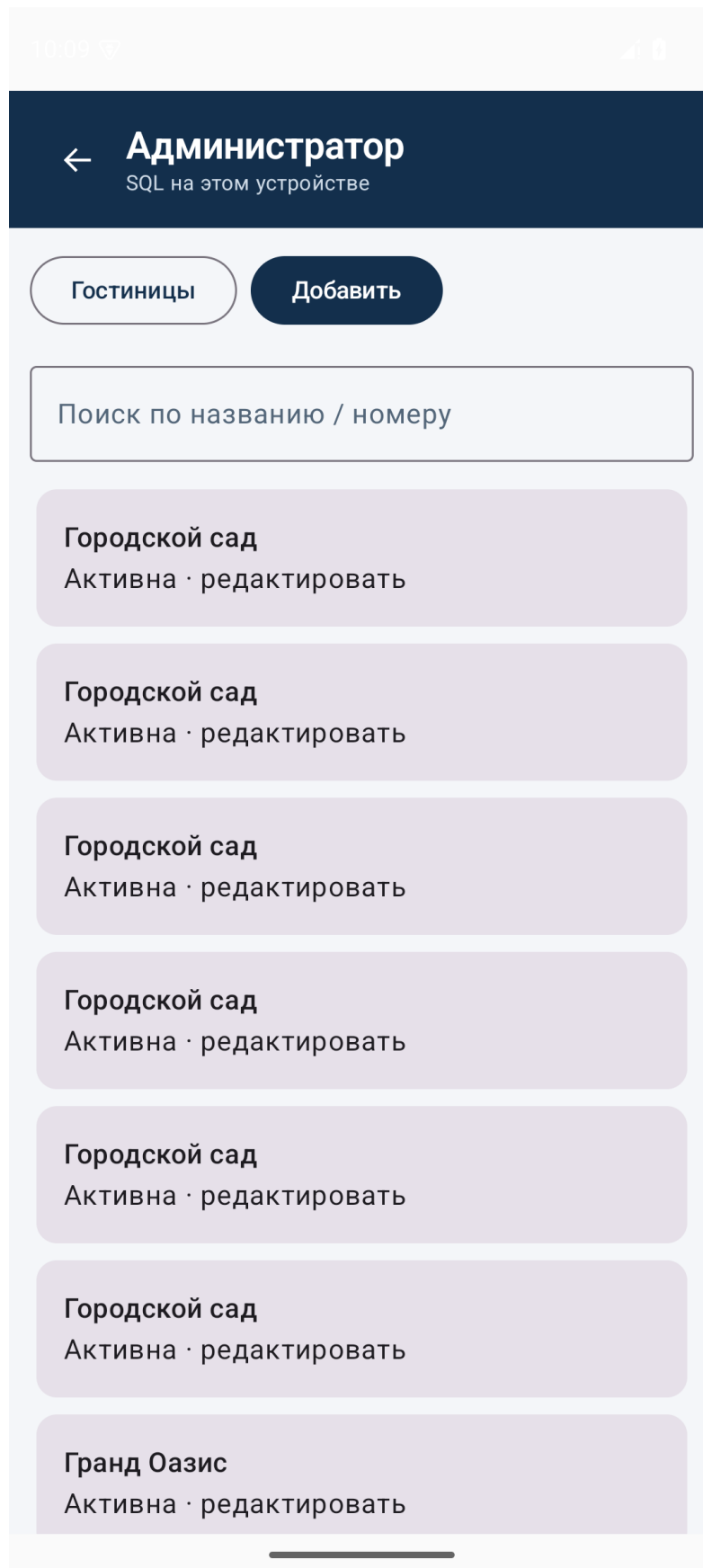


Рисунок П2.7 - Форма редактирования

10:09

← **Администратор**
Редактирование

Новая запись

Название

Город: Выбрать

Звёзды: 3

Демо-рейтинг
8.0

Адрес

Описание

Широта
0

Долгота
0

☒ Активна (выключить = архив)